Deephaven as an application framework: a ladder of examples

Each rung adds one idea. They run against the local server, started either with the top-level `docker-compose.yml` or with `server/run_server.py`. Paste a file into the IDE (http://localhost:10000/ide/) or use the driver:

```
.\.venv\Scripts\python -m pip install -r examples\requirements.txt   # once
.\.venv\Scripts\python examples\run.py examples\01_lifecycle.py --show ex1_ticks --every 2 --times 3
```

Rung	File	Idea	Your framework's analogue
1	`01_lifecycle.py`	The script runs once and returns; the table graph ticks forever on the update-graph clock	`EventManager.run()`, except here *you* don't own the loop
2	`02_declarative_dag.py`	Declared tables (bars, VWAP, rolling MA, signals) replace per-tick Python	The pipeline/DAG inside `on_data`
3	`03_listener_hook.py`	A listener receives each cycle's added/modified/removed rows (with previous values), mutates Python state, and writes to a sink table	`on_data` / `on_signal_update` + `self.state` + sink queues
4	`04_feeds_and_checkpoints.py`	Your asyncio tasks publish rows whenever they like; `merged_listen` delivers one consistent checkpoint per cycle across all feeds	Feed tasks + synchronizer
5	`05_introspection.py`	The engine's own profiler and dependency graph: per-node cost tables, ancestor graphs as DOT/SVG/deephaven.ui	Your diagnostics dashboard

Introspection (`05_introspection.py`, `dump_graph.py`)

- `perfmon.update_performance_log()` gives per-node, per-interval (about 1 min) usage/CPU nanos, invocation counts, rows added/modified/removed and allocations. Every table operation *and* every listener is a node. In our runs the two Python callbacks were the costliest nodes in the graph, about 195 ms per interval each, against about 80 ms for the rolling `update_by` and 3–36 ms for the other Java table operations.
- `perfmon.ancestor_dot / ancestor_svg / ancestor_image(entry_id, upl, ancestors_log)` draw everything upstream of a node. `examples/ex3_graph.svg` is rung 3's listener: 13 nodes back to the `TimeTable`, including `update_by`'s internal partitioning. Open `ex5_graph_ex3` / `ex5_graph_ex4` in the IDE or `/iframe/widget/?name=...`.
- Also available: `query_performance_log` / `query_operation_performance_log` (and `*_tree_table` variants) for how long table *creation* took, and `server_state_log` for cycle times, memory and GC.

- **Readable version:** `pretty_graph.py` (run via `dump_pretty.py`) redraws the same engine data with Python variable names, plain-language operations ("agg_by Sym: VWAP = wavg(Px, Qty) · …"), per-interval cost, downstream arrows, colour by kind, and engine internals collapsed. Roots can be table variable names, listener descriptions, or entry ids: `pretty_graph_image(["ex2_bars", "ex4 synchronizer"])`. The output is `examples/pretty_*.svg`, or `pretty_*_img` in the IDE.
- **Setup requirements we found:**
- The ancestors log only records dependencies for nodes created *after* it is first opened. `server/app.d/introspection.py` opens it at boot.
- SVG/image rendering needs the graphviz `dot` binary, which the stock image lacks; see `server/Dockerfile`.
- **Blind spots:** the graph shows engine nodes only. Your asyncio tasks, Python state and queues are invisible; a `table_publisher` feed shows up only as its output adapter. Graphs show ancestors of one node, not a whole-worker map, and there's no live view of in-flight cycles.

What the runs showed (observed, not assumed)

- **A checkpoint is an update cycle, about 1s here.** In rung 1, ticks scheduled 0.5s apart arrive in pairs with the same `Step`. In rung 4, each callback gets about 4 quotes, 4–5 orders and 1–2 signals together.
- **The latency floor is about one cycle.** Rows from asyncio sat buffered for up to about 1s (`MaxBufferedMs` 650–1000) before a checkpoint picked them up. The cycle length is configurable, but the default clock is shared by every table on the default update graph, which is where tables land unless you bind them to another graph. A worker can also run a separate **event-driven** update graph that advances only when your code calls `requestRefresh()`: three pushes stayed invisible until one refresh delivered them as a single step (`tests/probe_event_driven_clock.py`).
- **Reads inside a callback are consistent.** `OrdersTotal`, read from another table inside the callback, always agrees with the step being processed.
- **Recomputation is dependency-driven.** A formula that reads no columns is evaluated once per *added* row and never on modify (rung 3's first, stale state view).

Gotchas that matter for an application framework

1. **An exception in a listener stops it for good.** The engine logs a traceback and the callback never fires again. Feeds keep flowing and the worker still looks healthy. Wrap every hook (rung 4 records errors in `ex4_errors`).
2. `merged_listen` **passes every watched table each cycle**, with `None` for tables that didn't change.
3. **Re-running setup code stacks callbacks.** Rebinding `ex3_handle` does not stop the old listener. Stop the previous handle explicitly (rung 3).
4. **Background tasks must not read lifecycle state from globals.** Old asyncio tasks looked up `ex4_stop` by name, saw the next run's unset event, and ran forever. The leaked loops can't be reached afterwards; only a worker restart cleared them. Pass stop events (and anything else lifecycle-related) in explicitly (rung 4).
5. **Threads need the execution context, and contexts are per thread.** Capture `get_exec_ctx()` where the real context lives (inside `run_script`, as rung 4 does, or on the thread that started an embedded server) and enter it with `with ctx:` on your thread. Calling `get_exec_ctx()` on some other thread

returns a static-only context, and creating a ticking table there fails with `UpdateGraphConflictException` (`tests/embedded_thread_demo.py`).

6. **Map tables by identity**, not by `toString()` (rung 4's mislabeled first attempt).

Items 1, 3 and 4 are the real cost of putting a stateful framework *inside* the worker. Your `EventManager` would have to own this lifecycle discipline (handles, stop events, error capture) on top of the engine's.

Beyond the ladder

Later tests in `tests/` went further than rungs 1–5:

- `probe_runtime_features.py`: server executors (`submit_task`) and a second, event-driven update graph.
- `probe_event_driven_clock.py`: tables and a listener on that graph, advanced only by our own `requestRefresh()` calls.
- `host_service_demo.py`: our framework starting Deephaven as a subprocess and reading it at its own checkpoints.
- `embedded_thread_demo.py`: Deephaven on a background thread of our process, with in-process tables and listeners, GIL-contention measurements, and the one-server-per-process limit.

Next rungs

6. A mini `Algorithm` base class (`on_initialize`, `on_data`, `on_timer`, `on_command`, `on_shutdown`) on top of an event-driven graph our synchronizer steps.
7. Dynamic subscriptions: adding a feed from inside a callback (where the engine's locking rules bite) and the deferred-to-a-thread pattern.
8. A slow callback: measuring how much it stalls the other tables on its own update graph, and whether tables on a separate graph (own clock, same GIL) are affected.
9. A deephaven.ui front end for the algorithm's state and commands.